

Functions And Modular Code



Suman

27 Jan, 2026 At 9:00 PM

Pre-Reads

Trimester-1

Recommended

Resource



Associated Lectures (1)



Description



Discussions

Session 2.2: Functions & Modular Code

Pre-Read Materials

Program: AIM101 --- Artificial Intelligence & Machine Learning Foundation **Institution:** Vishlesan i-Hub IIT Patna x Masai School **Estimated Reading Time:** 15-20 minutes

What You Will Learn Today

By the end of Session 2.2, you will be able to:

- **Define** functions using the `def` keyword to encapsulate reusable code
- **Distinguish** between parameters (in function definition) and arguments (in function call)
- **Return** single values, multiple values, and handle `None` returns appropriately
- **Use** default parameter values and keyword arguments for flexible function calls
- **Understand** variable scope (local vs global) and why it matters
- **Write** professional docstrings to document your functions
- **Create** simple lambda functions for short, one-line operations
- **Import** and use Python's built-in modules (`math` , `random` , `datetime`)

Business Story: The E-Commerce Inventory Management Problem

The Scenario

You have been hired as a junior developer at "QuickMart," a fast-growing e-commerce startup. Your team manages an inventory system that tracks thousands of products across multiple warehouses.

The Code Nightmare

On your first day, you open the codebase and find this:

```
# Calculate discount for product 1
price1 = 1000
if price1 > 500:
    discount1 = price1 * 0.10
else:
    discount1 = price1 * 0.05
final_price1 = price1 - discount1
print(f"Product 1: Rs. {final_price1}")

# Calculate discount for product 2
price2 = 350
if price2 > 500:
    discount2 = price2 * 0.10
else:
    discount2 = price2 * 0.05
final_price2 = price2 - discount2
print(f"Product 2: Rs. {final_price2}")

# Calculate discount for product 3
price3 = 2500
if price3 > 500:
    discount3 = price3 * 0.10
else:
    discount3 = price3 * 0.05
final_price3 = price3 - discount3
print(f"Product 3: Rs. {final_price3}")

# ... repeated 500 more times for each product
```

The Problem

Your manager asks: "We need to change the discount threshold from Rs. 500 to Rs. 750, and the discount rates to 15% and 8%."

You realize you would need to:

- Find and modify **500+ copies** of the same logic
- Risk missing some occurrences (creating bugs)
- Spend hours on a 2-minute change

The Solution: Functions

What if the code looked like this instead?

```
def calculate_discount(price, threshold=500, high_rate=0.10, low_rate=0.05):
    """Calculate discount based on price threshold."""
    if price > threshold:
        discount = price * high_rate
    else:
        discount = price * low_rate
    return price - discount
```

```
# Now calculating discounts is simple
print(f"Product 1: Rs. {calculate_discount(1000)}")
print(f"Product 2: Rs. {calculate_discount(350)}")
print(f"Product 3: Rs. {calculate_discount(2500)}")
```

To change the discount rules, you modify **ONE function**. All 500 products automatically use the new logic. This is the power of **modular code**.

Real-World Impact

- **Netflix** uses functions to calculate personalized recommendations for 230 million users without duplicating code
- **Zomato/Swiggy** use functions to compute delivery times, restaurant ratings, and discount offers
- **Banks** use functions to process millions of transactions with consistent validation rules

Today's Session: You will learn to write functions that make your code reusable, maintainable, and professional.

Setup Requirements

Quick Environment Check (60 seconds)

Your environment should already be set up from previous sessions. Just verify:

Open Terminal/Command Prompt

Activate your environment:

```
conda activate mlenv
```

Your prompt should show `(mlenv)` at the beginning.

Check Python version:

```
python --version
```

Expected: Python 3.12.x

Launch Jupyter:

```
jupyter notebook
```

Expected: Browser opens with Jupyter interface

If any step fails, refer back to the Session 1.1 Pre-read for detailed troubleshooting instructions.

Concepts to Preview

1. Defining Functions

A function is a named block of code that performs a specific task. You define it once and call it whenever needed.

```
# Define a function
def greet(name):
    print(f"Hello, {name}!")

# Call the function
greet("Raj")    # Output: Hello, Raj!
greet("Priya")  # Output: Hello, Priya!
```

Key Points:

- `def` keyword starts a function definition
- Function name follows snake_case convention
- Parameters go inside parentheses
- Colon `:` ends the header line
- Function body must be indented (4 spaces)

2. Parameters vs Arguments

This distinction often confuses beginners:

```
def greet(name):    # 'name' is a PARAMETER (in definition)
    print(f"Hello, {name}!")

greet("Raj")        # "Raj" is an ARGUMENT (in call)
```

Analogy: Think of a parameter as a parking spot label, and an argument as the car that parks there.

3. Return Values

Functions can send values back to the calling code:

```
def add(a, b):
    return a + b    # Sends the result back

result = add(5, 3)  # result = 8
print(result * 2)   # Output: 16
```

Important: `return` is different from `print()`. Return gives the value back to your code; print just displays it on screen.

4. Default Parameter Values

You can give parameters default values that are used when no argument is provided:

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Raj")          # Output: Hello, Raj!
greet("Raj", "Welcome") # Output: Welcome, Raj!
```

5. Keyword Arguments

You can call functions using parameter names for clarity:

```
def create_profile(name, age, city):
    print(f"{name}, {age}, from {city}")

# Using keyword arguments (order doesn't matter)
create_profile(city="Delhi", name="Raj", age=22)
```

6. Variable Scope

Variables created inside a function are **local** --- they exist only within that function:

```
def my_function():
    x = 10          # Local variable
    print(x)

my_function()      # Output: 10
print(x)           # ERROR! x doesn't exist outside the function
```

7. Docstrings

Docstrings document what a function does. They appear right after the function definition:

```
def calculate_area(length, width):
    """
    Calculate the area of a rectangle.

    Parameters:
        length: The length of the rectangle
        width: The width of the rectangle

    Returns:
        The area of the rectangle
    """
    return length * width
```

8. Lambda Functions

Lambda functions are small, anonymous functions for simple operations:

```
# Regular function
def square(x):
    return x * x

# Equivalent lambda function
square = lambda x: x * x

print(square(5)) # Output: 25
```

9. Importing Modules

Python has built-in modules with pre-written functions:

```
import math
print(math.sqrt(16)) # Output: 4.0

import random
print(random.randint(1, 10)) # Random number between 1 and 10

from datetime import datetime
print(datetime.now()) # Current date and time
```

Key Vocabulary

Familiarize yourself with these terms before class:

Term	Definition
function	A named, reusable block of code that performs a specific task
def	The keyword used to define a function in Python
parameter	A variable in a function definition that receives a value
argument	The actual value passed to a function when calling it
return	A statement that sends a value back from a function to the caller
None	The value returned by a function that has no return statement
default parameter	A parameter with a pre-assigned value used if no argument is provided
keyword argument	An argument passed using the parameter name (e.g., <code>name="Raj"</code>)
positional argument	An argument matched to a parameter by its position in the call
scope	The region of code where a variable is accessible
local variable	A variable defined inside a function, accessible only within it
global variable	A variable defined outside all functions, accessible everywhere

Term	Definition
docstring	A string that documents what a function does, placed right after <code>def</code>
lambda	A keyword for creating small, anonymous functions in one line
module	A file containing Python code (functions, variables) that can be imported
import	A statement that loads a module so you can use its functions

Things to Ponder

Think about these questions before class. We will explore them during the session.

1. When Should You Create a Function?

If you find yourself copying and pasting the same code more than twice, is that a sign you need a function? What other signals indicate that code should be wrapped in a function?

2. Return vs Print: Why Does It Matter?

Consider these two versions:

```
def add_print(a, b):  
    print(a + b)  
  
def add_return(a, b):  
    return a + b
```

Which one would you use if you needed to:

- Store the result for later use?
- Use the result in another calculation?
- Display the result to the user?

3. Why Does Scope Exist?

Why doesn't Python just make all variables accessible everywhere? What problems could arise if there were no local and global scope distinction?

4. When Would You Use Lambda Functions?

Lambda functions are limited to one expression. When would this limitation be acceptable? When would a regular function be better?

Optional Deep Dive

For students who want to explore further before class:

Video Resources

- **Corey Schafer: Python Functions** (YouTube, ~20 min) - Clear explanation of function basics
- **Real Python: Defining Your Own Functions** - Comprehensive written tutorial

Documentation

- **Python Official Docs: Defining Functions** - <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>
- **PEP 257: Docstring Conventions** - <https://peps.python.org/pep-0257/>

Practice Exercises (Optional)

Try writing these functions before class:

A function that takes a temperature in Celsius and returns it in Fahrenheit

A function that takes a list of numbers and returns the average

A function that takes a name and an optional greeting, and prints a personalized message

Previous Sessions Recap: What You Already Know

Before Session 2.2, make sure you are comfortable with these concepts from Sessions 1.1 through 2.1:

Variables and Data Types

```
name = "Priya"      # str - text
age = 25             # int - whole number
height = 5.6         # float - decimal
is_student = True    # bool - True/False
```

Control Flow

```
if age >= 18:
    print("Adult")
elif age >= 13:
    print("Teenager")
else:
    print("Child")
```

Loops

```
for i in range(5):
    print(i)          # Prints 0, 1, 2, 3, 4
```



```
while count < 10:
    count += 1
```

Data Structures

```
# Lists - ordered, mutable
fruits = ["apple", "banana", "cherry"]

# Tuples - ordered, immutable
coordinates = (10, 20)

# Dictionaries - key-value pairs
student = {"name": "Raj", "age": 22}

# Sets - unique items
colors = {"red", "green", "blue"}
```

Pre-Class Checklist

Before coming to class, ensure you have completed the following:

- ☐ Read this entire document
- ☐ Reviewed concepts from Sessions 1.1 through 2.1
- ☐ Verified your Python environment works (`conda activate mlenv`)
- ☐ Launched Jupyter Notebook successfully
- ☐ Understood the basic syntax for defining a function
- ☐ Know the difference between parameters and arguments
- ☐ Understand why `return` is different from `print()`
- ☐ Reviewed the key vocabulary above
- ☐ Thought about the "Things to Ponder" questions
- ☐ Practiced the preview code examples in a Jupyter notebook (optional but recommended)

Quick Reference: Function Anatomy

```
def function_name(parameter1, parameter2="default"):
    """
    Docstring explaining what the function does.
    """
    # Function body (indented)
    result = parameter1 + parameter2
    return result

# Calling the function
output = function_name(argument1, argument2)
```

Component	Description
<code>def</code>	Keyword to start function definition
<code>function_name</code>	Name you choose (use snake_case)
<code>parameters</code>	Inputs the function accepts
<code>:</code>	Required colon after the header
<code>docstring</code>	Optional documentation string
<code>body</code>	Indented code that runs when called
<code>return</code>	Optional statement to send value back
<code>arguments</code>	Actual values passed when calling

See You in Class!

Today you will learn to organize code into reusable functions, making your programs cleaner, more maintainable, and more professional. Functions are the building blocks of all serious programming --- mastering them is essential for your journey into data science and machine learning.

Get ready to transform repetitive code into elegant, modular solutions!

Vishlesan i-Hub IIT Patna × Masai School